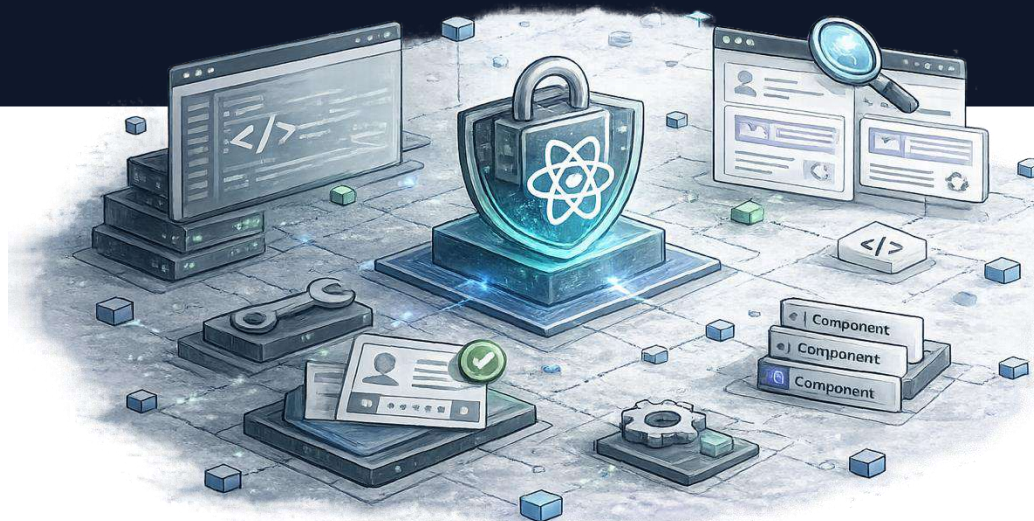




Progettazione di applicazioni web full stack Laboratorio su React



Prof. Devis Bianchini
Dip. di Università degli Studi di Brescia



Argomento di questa lezione

Cosa faremo

- Creazione di una *pagina front-end* in **React** per visualizzare un catalogo di libri, mettendo in comunicazione *front-end* e *back-end*
- Proteggeremo la pagina «dietro» il meccanismo di *autenticazione* e *autorizzazione*

Output

- Esempi di pagine sviluppate in **React** e *relative rotte*
- Meccanismo di interazione tra *front-end* e *back-end* che includa anche autenticazione e autorizzazione

Procedura che seguiremo



1. Predisponiamo una funzionalità di backend per l'estrazione del catalogo di libri
2. Abilitiamo la comunicazione tra front-end e back-end
3. Predisponiamo rotte e pagine nell'applicazione di front-end



- File `books.service.ts`



Estraiamo il catalogo dei libri dal database



```
async findAllBooks(): Promise<BookListItem[]> {  
  const books = await this.bookRepository.find({  
    relations: {  
      authors: true,  
      category: true  
    },  
    order: {  
      title: 'ASC'  
    }  
  });  
  
  return books.map((book) => ({  
    id: book.id,  
    title: book.title,  
    publishedYear: book.publishedYear,  
    category: {  
      id: book.category.id,  
      name: book.category.name  
    },  
    authors: book.authors.map((author) => ({  
      id: author.id,  
      firstName: author.firstName,  
      lastName: author.lastName  
    })))  
  }  
});
```



- File `books.service.ts`

Definiamo un'interfaccia per la comunicazione tra front-end e back-end:

```
export interface BookListItem {  
  id: number;  
  title: string;  
  publishedYear: number;  
  category: {  
    id: number;  
    name: string;  
  };  
  authors: {  
    id: number;  
    firstName: string;  
    lastName: string;  
  }[];  
}
```

```
async findAllBooks(): Promise<BookListItem[]> {  
  const books = await this.bookRepository.find({  
    relations: {  
      authors: true,  
      category: true  
    },  
    order: {  
      title: 'ASC'  
    }  
  });  
  
  return books.map((book) => ({  
    id: book.id,  
    title: book.title,  
    publishedYear: book.publishedYear,  
    category: {  
      id: book.category.id,  
      name: book.category.name  
    },  
    authors: book.authors.map((author) => ({  
      id: author.id,  
      firstName: author.firstName,  
      lastName: author.lastName  
    })))  
  }));  
}
```



- File `books.controller.ts`



Estraiamo il catalogo dei libri dal database



```
import { Controller, Get, Post, UseGuards } from '@nestjs/common';
import { OrgBooksService } from '../books.service';
import { ApiTags, ApiBearerAuth } from '@nestjs/swagger';
import { JwtAuthGuard, Roles, RolesGuard } from '@server/security';
import { UserRole } from '@server/users';
import { BookListItem } from '../interfaces/book-list-item.interface';

@ApiTags('Library APIs')
@Controller('books')
export class OrgBooksController {
  constructor(private orgBooksService: OrgBooksService) {}

  @Post('populate')
  populateDB() {
    return this.orgBooksService.seed();
  }

  @Get()
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles(UserRole.ADMIN)
  @ApiBearerAuth()
  findAll(): Promise<BookListItem[]> {
    return this.orgBooksService.findAllBooks();
  }
}
```

Cross-Origin Resource Sharing



- File `main.ts` nell'applicazione di back-end



Estraiamo il catalogo dei libri dal database



Abilitiamo la comunicazione tra front-end e back-end



```
async function bootstrap() {  
  const app = await NestFactory.create(AppModule);  
  
  app.enableCors({  
    origin: 'http://localhost:4200',  
    credentials: true,  
    allowedHeaders: ['Content-Type', 'Authorization'],  
    methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS']  
  });  
  
  const globalPrefix = 'api';  
  app.setGlobalPrefix(globalPrefix);  
}
```

Cross-Origin Resource Sharing



- File `main.tsx` nell'applicazione di back-end

Permette a React su `localhost:4200` di chiamare NestJS su `localhost:3333`, mandando JSON e token JWT nell'header **Authorization**

- Utilizzando delle credenziali (cookie, header di autenticazione, certificati)
- Usando gli header **Content-Type** (per es. per la login) e **Authorization** (JWT) nella richiesta HTTP
- Utilizzando i metodi HTTP indicati

```
async function bootstrap() {  
  const app = await NestFactory.create(AppModule);  
  
  app.enableCors({  
    origin: 'http://localhost:4200',  
    credentials: true,  
    allowedHeaders: ['Content-Type', 'Authorization'],  
    methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS']  
  });  
  
  const globalPrefix = 'api';  
  app.setGlobalPrefix(globalPrefix);  
}
```


Front-end (I)



- Componente **app.tsx**



Estraiamo il catalogo dei libri dal database



Abilitiamo la comunicazione tra front-end e back-end



Modifichiamo il componente principale del front-end



```
import { BooksPage } from '../features/books/books.page';
import { Routes, Route, Navigate } from 'react-router-dom';
import { LoginPage } from '../features/auth/login.page';

export function App() {
  return (
    <Routes>
      <Route path="/" element={<Navigate to="/books" replace />} />
      <Route path="/login" element={<LoginPage />} />
      <Route path="/books" element={<BooksPage />} />
    </Routes>
  );
}

export default App;
```

Front-end (II)



- File `auth.api.ts`



Estraiamo il catalogo dei libri dal database



Abilitiamo la comunicazione tra front-end e back-end



Modifichiamo il componente principale del front-end



Predisponiamo le rotte per il catalogo e la login

```
const API_URL = 'http://localhost:3333/api';

export async function login(email: string, password: string) {
  const response = await fetch(`${API_URL}/auth/login`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({ email, password }),
  });

  if (!response.ok) {
    throw new Error('Credenziali non valide');
  }

  const data = await response.json();

  localStorage.setItem('access_token', data.access_token);

  return data;
}
```



- File `books.api.ts`



Estraiamo il catalogo dei libri dal database



Abilitiamo la comunicazione tra front-end e back-end



Modifichiamo il componente principale del front-end



Predisponiamo le rotte per il catalogo e la login

```
import { BookListItem } from './types';

const API_URL = 'http://localhost:3333/api';

export async function fetchBooks(): Promise<BookListItem[]> {
  const token = localStorage.getItem('access_token');

  const response = await fetch(`${API_URL}/books`, {
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

  if (!response.ok) {
    throw new Error('Errore durante il caricamento dei libri');
  }

  return response.json();
}
```

Pagina per login (I)



```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { login } from './auth.api';
import book_styles from '../css/books.module.css';

export function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);

  const navigate = useNavigate();

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setError(null);
    setLoading(true);

    try {
      await login(email, password);
      navigate('/books');
    } catch (err: any) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }
}
```

Pagina per login (II)



- Rendering della pagina di login (con stile)

- Richiede di aver definito e importato un foglio di stile

```
/* ===== Layout generale ===== */
.page {
  min-height: 100vh;
  background: #f5f7fa;
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 2rem;
  box-sizing: border-box;
}
```

```
import book_styles from '../css/books.module.css';
```

```
return (
  <main className={book_styles.page}>
    <div className={book_styles.card}>
      <h1 className={book_styles.title}>📖 Library Login</h1>

      <form onSubmit={handleSubmit} className={book_styles.form}>
        <div className={book_styles.field}>
          <label>Email</label>
          <input
            className={book_styles.input}
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            placeholder="Inserisci email"
            required
          />
        </div>
      </form>
    </div>
  </main>
)
```

Pagina per login (III)



```
<div className={book_styles.field}>
  <label>Password</label>
  <input
    type="password"
    className={book_styles.input}
    value={password}
    onChange={(e) => setPassword(e.target.value)}
    placeholder="Inserisci password"
    required
  />
</div>

<button className={book_styles.button} type="submit" disabled={loading}>
  {loading ? 'Accesso in corso...' : 'Login'}
</button>
</form>

{error && <p className={book_styles.error}>{error}</p>}
</div>
</main>
);
```

Pagina per la visualizzazione del catalogo (I)



- Foglio di stile condiviso tra le diverse pagine

```
import { useEffect, useState } from 'react';
import { fetchBooks } from './books.api';
import { BookListItem } from './types';
import { useNavigate } from 'react-router-dom';
import book_styles from '../css/books.module.css';
import clsx from 'clsx';

export function BooksPage() {
  const [books, setBooks] = useState<BookListItem[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const navigate = useNavigate();

  useEffect(() => {
    fetchBooks()
      .then(setBooks)
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, []);

  if (loading) {
    return (
      <main className={book_styles.page}>
        <div className={book_styles.card}>
          <p className={book_styles.message}>Caricamento libri...</p>
        </div>
      </main>
    );
  }
}
```


Pagina per la visualizzazione del catalogo (I)



- Foglio di stile condiviso tra le diverse pagine

- Pacchetto comodo per applicare diverse regole di stile

```
import { useEffect, useState } from 'react';
import { fetchBooks } from './books.api';
import { BookListItem } from './types';
import { useNavigate } from 'react-router-dom';
import book_styles from '../css/books.module.css';
import clsx from 'clsx';

export function BooksPage() {
  const [books, setBooks] = useState<BookListItem[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const navigate = useNavigate();

  useEffect(() => {
    fetchBooks()
      .then(setBooks)
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, []);

  if (loading) {
    return (
      <main className={book_styles.page}>
        <div className={book_styles.card}>
          <p className={book_styles.message}>Caricamento libri...</p>
        </div>
      </main>
    );
  }
}
```


Pagina per la visualizzazione del catalogo (II)



- Rendering della pagina con il catalogo di libri (con stile)

- Applicazione di regole multiple grazie al pacchetto **clsx**

```
/* ===== Card riutilizzabile ===== */
.card {
  background: white;
  padding: 2rem;
  border-radius: 12px;
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
}

/* Card larga (catalogo) */
.cardLarge {
  width: 100%;
  max-width: 1000px;
}

/* Card stretta (login) */
.cardSmall {
  width: 100%;
  max-width: 360px;
}
```

```
return (
  <main className={book_styles.page}>
    <section className={clsx(book_styles.card, book_styles.cardLarge)}>
      <header className={book_styles.headerRow}>
        <div>
          <h1 className={book_styles.title}>📖 Catalogo libri</h1>
          <p className={book_styles.subtitle}>
            Elenco dei libri disponibili nella libreria online.
          </p>
        </div>

        <button
          className={book_styles.logoutButton}
          onClick={() => navigate('/logout')}
        >
          Logout
        </button>
      </header>
    </section>
  </main>
)
```

Pagina per la visualizzazione del catalogo (II)



- Rendering della pagina con il catalogo di libri (con stile)

- Applicazione di regole multiple grazie al pacchetto **clsx**

```
/* ===== Card riutilizzabile ===== */
.card {
  background: white;
  padding: 2rem;
  border-radius: 12px;
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
}

/* Card larga (catalogo) */
.cardLarge {
  width: 100%;
  max-width: 1000px;
}

/* Card stretta (login) */
.cardSmall {
  width: 100%;
  max-width: 360px;
}
```

```
return (
  <main className={book_styles.page}>
    <section className={clsx(book_styles.card, book_styles.cardLarge)}>
      <header className={book_styles.headerRow}>
        <div>
          <h1 className={book_styles.title}>📖 Catalogo libri</h1>
          <p className={book_styles.subtitle}>
            Elenco dei libri disponibili nella libreria online.
          </p>
        </div>

        <button
          className={book_styles.logoutButton}
          onClick={() => navigate('/logout')}
        >
          Logout
        </button>
      </header>
    </section>
  </main>
)
```

Pulsante di **Logout** (vedi slide successiva)

Pagina per la visualizzazione del catalogo (III)



```
{books.length === 0 ? (  
  <p className={book_styles.message}>Nessun libro disponibile.</p>  
) : (  
  <div className={book_styles.tableWrapper}>  
    <table className={book_styles.table}>  
      <thead>  
        <tr>  
          <th className={book_styles.th}>Titolo</th>  
          <th className={book_styles.th}>Anno</th>  
          <th className={book_styles.th}>Categoria</th>  
          <th className={book_styles.th}>Autori</th>  
        </tr>  
      </thead>  
  
      <tbody>  
        {books.map((book) => (  
          <tr key={book.id} className={book_styles.row}>  
            <td className={book_styles.titleCell}>{book.title}</td>  
            <td className={book_styles.td}>{book.publishedYear}</td>  
            <td className={book_styles.td}>  
              <span className={book_styles.badge}>  
                {book.category?.name ?? 'N/D'}  
              </span>  
            </td>  
          </tr>  
        )  
      )  
    </tbody>  
  </div>  
)
```

Pagina per la visualizzazione del catalogo (IV)



```
        <td className={book_styles.td}>
          {book.authors?.length
            ? book.authors
              .map(
                (author) =>
                  `${author.firstName} ${author.lastName}`
              )
              .join(', ')
            : 'N/D'}
        </td>
      </tr>
    )}
  </tbody>
</table>
</div>
)}
</section>
</main>
```

Componente per il logout



- È gestito totalmente all'interno dell'applicazione React, non richiede comunicazione con il *back-end* (viene eliminato il token JWT)

```
// features/auth/logout.page.tsx
import { useEffect } from 'react';
import { useNavigate } from 'react-router-dom';

export function LogoutPage() {
  const navigate = useNavigate();

  useEffect(() => {
    // 1. rimuovi token
    localStorage.removeItem('access_token');

    // 2. redirect
    navigate('/login', { replace: true });
  }, [navigate]);

  return <p>Logout in corso...</p>;
}
```

Proteggere una rotta (I)



- Utile un componente aggiuntivo, che implementa il middleware

```
import { ReactNode } from 'react';
import { Navigate } from 'react-router-dom';

export function ProtectedRoute({ children }: { children: ReactNode }) {
  const token = localStorage.getItem('access_token');

  if (!token) {
    return <Navigate to="/login" replace />;
  }

  return children;
}
```

Proteggere una rotta (II)



- Utile un componente aggiuntivo, che implementa il middleware

```
import { ReactNode } from 'react';
import { Navigate } from 'react-router-dom';

export function ProtectedRoute({ children }) {
  const token = localStorage.getItem('token');

  if (!token) {
    return <Navigate to="/login" replace />;
  }

  return children;
}

export function App() {
  return (
    <Routes>
      <Route path="/" element={<Navigate to="/books" replace />} />
      <Route path="/login" element={<LoginPage />} />
      <Route
        path="/books"
        element={
          <ProtectedRoute>
            <BooksPage />
          </ProtectedRoute>
        }
      />
      <Route path="/logout" element={<LogoutPage />} />
    </Routes>
  );
}
```

- Modifica del file `app.tsx`



Progettazione di applicazioni web full stack Laboratorio su React



Prof. Devis Bianchini
Dip. di Università degli Studi di Brescia